Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

I added to the code, using a for loop to create floating-coloured balls that on impact, changed the blob colours to whatever colour the ball was. And the ball disappears. It was easier figuring out how to make tis idea come to life using AI.

Role-Based Process Evidence

Entry Header

Name: Victory Okusanya

Primary responsibility for this work:

Goal of Work Session

Goal was to use a for loop to help create a constant image/tool in the code.

Tools, Resources, or Inputs Used

- GenAI tools
- Lecture Notes
- Classmate comprehension
- Prior drafts or code

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: February 13, 2026

Tool Disclosure: Claude ai

Purpose of Use: Using the ai to figure out how to create colour balls that turned changed the colour of the blob when it came in contact. And learn how to change between levels.

Summary of Interaction: Asked about the requirements and features I wanted to add, describing the js I need to put the code in. I inputted json files with those codes and Claude

told me what to add, and provided a description of what I need to add and provided an example.

Human Decision Point(s): I agreed with a lot of the decisions, expect how far the colours reached past the level/platforms so I changed it's constraints. I also changed the platform location for the 2nd level, and a couple edits for the 1st level. I chose how and what the for loop looked like a designed.

I also asked about code to change between the levels and omitted whatever seemed unnecessary and was.

Integrity & Verification Note: I play tested the code and changed any visual issues I saw.

Scope of GenAI Use: GenAI contributed to the level.js, sketch.js, and WORDlevel.js, and didn't aid in other script or levels.

Limitations or Misfires: The GenAi was pretty accurate, just had a hard time grasping the length(range) of the level/game.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

- Used ai to help with coding knowledge gap.
- Thought of plan and design provocative.
- Coded.

Appendix

ME: Hi, I have these 3 js scripts that called: Levels.js (in charge of level/platform creation), WorldLevel.js(turns levels.json into usable object), and sketch.js(draws the blob, in charge of drawing the script). Can you help add code that will include a for loop for the levels that has floating circle balls that if the ball touches, changes into that colour?

CLAUDE:

'll add color-changing collectible balls that are generated with a for loop from data defined in levels.json. Here's the plan:

- Add a colorBalls config to each level in levels.json (list of colors)
- In WorldLevel.js, use a for loop to generate and space the balls across the world, and add a drawBalls() method

- In sketch.js, check collision each frame and change the blob's color on touch

- Let's make the changes:
- Orchestrated file modifications and code implementation strategy
- Read levels.json to edit it
- Now let me make all the changes. I'll copy the files to /home/claude, modify them, and present them.
- Done
- Now I have a clear picture. Let me make all three edits:
- Copy project files to working directory
- Copy project files to working directory
- Script
- Done
- **1. `levels.json`** — add a `colorBalls` block to each level:
- Edited 2 files
- Add colorBalls config to level 1
- levels.json+9-4
- Add colorBalls config to level 2
- levels.json+11-6
- Done
- **2. `WorldLevel.js`** — generate balls with a `for` loop and add `drawBalls()`:
- Add ball generation for loop and drawBalls method to WorldLevel
- Add ball generation for loop and drawBalls method to WorldLevel
- WorldLevel.js+42-7
- Done
- **3. `sketch.js`** — draw balls, check collisions, and change the blob color:
- Edited 3 files

Levels


```
{
  "schemaVersion": 1,
  "levels": [
    {
      "name": "level 1",
      "theme": {
        "bg": "#F0F0F0",
        "platform": "#C8C8C8",
        "blob": "#1478FF"
```

```
},
"gravity": 0.65,
"jumpV": -11,
"camera": {
  "lerp": 0.12
},
"world": {
  "w": 2400,
  "h": 360,
  "deathY": 560
},
"start": {
  "x": 80,
  "y": 220,
  "r": 26
},
"platforms": [
  {
    "x": 0,
    "y": 424,
    "w": 640,
    "h": 36
  },
  {
    "x": 130,
    "y": 354,
```

"w": 110,
"h": 12
},
{
"x": 300,
"y": 284,
"w": 110,
"h": 12
},
{
"x": 480,
"y": 354,
"w": 110,
"h": 12
},
{
"x": 620,
"y": 414,
"w": 80,
"h": 12
},
{
"x": 720,
"y": 424,
"w": 420,
"h": 36

},
{
"x": 770,
"y": 354,
"w": 110,
"h": 12
},
{
"x": 950,
"y": 284,
"w": 110,
"h": 12
},
{
"x": 1060,
"y": 354,
"w": 90,
"h": 12
},
{
"x": 1200,
"y": 424,
"w": 520,
"h": 36
},
{

```
    "x": 1260,
    "y": 354,
    "w": 120,
    "h": 12
  },
  {
    "x": 1440,
    "y": 284,
    "w": 120,
    "h": 12
  },
  {
    "x": 1600,
    "y": 354,
    "w": 120,
    "h": 12
  }
],
"colorBalls": {
  "r": 14,
  "floatY": 380,
  "colors": [
    "#FF4444",
    "#FF9900",
    "#44CC44",
    "#CC44FF",
```

```
      "#00CCFF"
    ]
  }
},
{
  "name": "level 2",
  "theme": {
    "bg": "#F0F0F0",
    "platform": "#C8C8C8",
    "blob": "#1478FF"
  },
  "gravity": 0.65,
  "jumpV": -11,
  "camera": {
    "lerp": 0.12
  },
  "world": {
    "w": 2400,
    "h": 360,
    "deathY": 560
  },
  "start": {
    "x": 80,
    "y": 220,
    "r": 26
  },
```

"platforms": [
 {
  "x": 0,
  "y": 424,
  "w": 640,
  "h": 36
 },
 {
  "x": 130,
  "y": 354,
  "w": 110,
  "h": 12
 },
 {
  "x": 300,
  "y": 284,
  "w": 110,
  "h": 12
 },
 {
  "x": 480,
  "y": 354,
  "w": 110,
  "h": 12
 },
 {

"x": 620,
"y": 414,
"w": 80,
"h": 12
},
{
"x": 720,
"y": 424,
"w": 420,
"h": 36
},
{
"x": 770,
"y": 354,
"w": 110,
"h": 12
},
{
"x": 950,
"y": 284,
"w": 110,
"h": 12
},
{
"x": 1060,
"y": 354,

"w": 90,
"h": 12
},
{
"x": 1200,
"y": 424,
"w": 520,
"h": 36
},
{
"x": 1260,
"y": 354,
"w": 120,
"h": 12
},
{
"x": 1440,
"y": 284,
"w": 120,
"h": 12
},
{
"x": 1600,
"y": 354,
"w": 120,
"h": 12

```
      }
    ],
    "colorBalls": {
      "r": 14,
      "floatY": 380,
      "colors": [
        "#FF4444",
        "#FF9900",
        "#44CC44",
        "#CC44FF",
        "#00CCFF"
      ]
    }
  }
 ]
}
```


Wordlevel

```
class WorldLevel {
  constructor(levelJson) {
    this.name = levelJson.name ?? "Level";


    this.theme = Object.assign(
      { bg: "#F0F0F0", platform: "#C8C8C8", blob: "#1478FF" },
      levelJson.theme ?? {},
    );
```

```
// Physics knobs
this.gravity = levelJson.gravity ?? 0.65;
this.jumpV = levelJson.jumpV ?? -11.0;

// Camera knob (data-driven view state)
this.camLerp = levelJson.camera?.lerp ?? 0.12;

// World size + death line
this.w = levelJson.world?.w ?? 2400;
this.h = levelJson.world?.h ?? 360;
this.deathY = levelJson.world?.deathY ?? this.h + 200;

// Start
this.start = Object.assign({ x: 80, y: 220, r: 26 }, levelJson.start ?? {});

// Platforms
this.platforms = (levelJson.platforms ?? []).map(
  (p) => new Platform(p.x, p.y, p.w, p.h),
);

// Color balls — generated with a for loop, evenly spaced across the world
this.balls = [];
if (levelJson.colorBalls) {
  const cfg = levelJson.colorBalls;
  const colors = cfg.colors;
  const ballR = cfg.r ?? 14;
```

```
    const floatY = cfg.floatY ?? 380;

    for (let i = 0; i < colors.length; i++) {
      // Space balls evenly: divide world width into (count + 1) slots
      const bx = (this.w / (colors.length + 1)) * (i + 1);
      this.balls.push({
        x: bx,
        y: floatY,
        r: ballR,
        col: colors[i],
        collected: false,
      });
    }
  }
}

drawBalls(t) {
  noStroke();
  for (const b of this.balls) {
    if (b.collected) continue;
    // gentle up-and-down float using sine and the sketch time
    const floatOffset = sin(t * 2 + b.x * 0.01) * 6;
    fill(b.col);
    circle(b.x, b.y + floatOffset, b.r * 2);
    // white shine dot
    fill(255, 255, 255, 160);
```

```
    circle(b.x - b.r * 0.3, b.y + floatOffset - b.r * 0.3, b.r * 0.6);
  }
}

drawWorld() {
  background(this.theme.bg);
  push();
  rectMode(CORNER);        // critical: undo any global rectMode(CENTER) [web:230]
  noStroke();
  fill(this.theme.platform);

  for (const p of this.platforms) rect(p.x, p.y, p.w, p.h); // x,y = top-left [web:234]
  pop();
}
}
```

Sketch

```
/*
Week 5 â€” Example 5: Side-Scroller Platformer with JSON Levels + Modular Camera

Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
Date: Feb. 12, 2026

Move: WASD/Arrows | Jump: Space

Learning goals:
```

```
- Build a side-scrolling platformer using modular game systems
- Load complete level definitions from external JSON (LevelLoader + levels.json)
- Separate responsibilities across classes (Player, Platform, Camera, World)
- Implement gravity, jumping, and collision with platforms
- Use a dedicated Camera2D class for smooth horizontal tracking
- Support multiple levels and easy tuning through data files
- Explore scalable project architecture for larger games
*/

const VIEW_W = 800;
const VIEW_H = 480;

let allLevelsData;
let levelIndex = 0;

let level;
let player;
let cam;
let playerColor; // tracks current blob colour, changes on ball pickup

function preload() {
  allLevelsData = loadJSON("levels.json"); // levels.json beside index.html [web:122]
}

function setup() {
  createCanvas(VIEW_W, VIEW_H);
```

```
  textFont("sans-serif");
  textSize(14);

  cam = new Camera2D(width, height);
  loadLevel(levelIndex);
}

function loadLevel(i) {
  level = LevelLoader.fromLevelsJson(allLevelsData, i);

  player = new BlobPlayer();
  player.spawnFromLevel(level);
  playerColor = level.theme.blob; // reset colour to level default on (re)load

  cam.x = player.x - width / 2;
  cam.y = 0;
  cam.clampToWorld(level.w, level.h);
}

function draw() {
  // --- game state ---
  player.update(level);

  // Fall death â†' respawn
  if (player.y - player.r > level.deathY) {
    loadLevel(levelIndex);
```

```
  return;
}

// --- view state (data-driven smoothing) ---
cam.followSideScrollerX(player.x, level.camLerp);
cam.y = 0;
cam.clampToWorld(level.w, level.h);

// --- draw ---
cam.begin();
level.drawWorld();
level.drawBalls(frameCount * 0.05);  // pass time for floating animation

// Check if player touched any uncollected ball
for (const b of level.balls) {
  if (!b.collected) {
    const d = dist(player.x, player.y, b.x, b.y);
    if (d < player.r + b.r) {
      b.collected = true;       // remove ball from world
      playerColor = b.col;      // change blob to ball's colour
    }
  }
}

player.draw(playerColor);
cam.end();
```

```
  // HUD
  fill(0);
  noStroke();
  text(level.name + " (Example 5)", 10, 18);
  text("A/D or â†/â†’ move â€¢ Space/W/â†‘ jump â€¢ Fall = respawn", 10, 36);
  text("camLerp(JSON): " + level.camLerp + "  world.w: " + level.w, 10, 54);
  text("cam: " + cam.x + ", " + cam.y, 10, 90);
  const p0 = level.platforms[0];
  text(` p0: x=${p0.x} y=${p0.y} w=${p0.w} h=${p0.h}` , 10, 108);

  text(
    "platforms: " +
      level.platforms.length +
      " start: " +
      level.start.x +
      "," +
      level.start.y,
    10,
    72,
  );
}

function keyPressed() {
  if (key === " " || key === "W" || key === "w" || keyCode === UP_ARROW) {
    player.tryJump();
```

```
  }
  if (key === "r" || key === "R") loadLevel(levelIndex);
}
```

ME: Does this code include information that makes it possible switch between levels? if not how an I do that?

CALUDE:

Examined existing level infrastructure and identified missing advancement mechanism

Looking at your code, the infrastructure is *almost* there but level switching isn't actually wired up. Here's what you have vs. what's missing:

Already exists:

- allLevelsData loads both levels from the JSON
- levelIndex = 0 tracks which level you're on
- loadLevel(i) can load any level by index

Missing:

- Nothing ever changes levelIndex or calls loadLevel with a different index — so you're permanently stuck on level 0

The simplest approach for a platformer is a finish zone at the end of each level that triggers the switch. Here's how to add it:

In levels.json, add a finish object to each level:

```
json
"finish": { "x": 1700, "y": 350, "w": 40, "h": 80 }
```

In WorldLevel.js, store and draw it in drawWorld():

```
js
// in constructor
this.finish = levelJson.finish ?? null;
```

```
// new method
drawFinish() {
  if (!this.finish) return;
  const f = this.finish;
  fill("#FFD700");
  noStroke();
  rect(f.x, f.y, f.w, f.h);
}
```

In sketch.js, check if the player overlaps the finish zone each frame and advance the level:

js

```
// inside draw(), after player.update(level)
if (level.finish) {
  const f = level.finish;
  if (player.x + player.r > f.x && player.x - player.r < f.x + f.w &&
      player.y + player.r > f.y && player.y - player.r < f.y + f.h) {
    levelIndex = (levelIndex + 1) % allLevelsData.levels.length; // wraps back to 0
    loadLevel(levelIndex);
    return;
  }
}
```

You could also just press a key — e.g. add this to keyPressed() for quick testing:

js

```
if (key === "n" || key === "N") {
  levelIndex = (levelIndex + 1) % allLevelsData.levels.length;
  loadLevel(levelIndex);
```

}

The finish zone approach is more game-like and fits the platformer structure you already have. Want me to go ahead and implement it?